INTELLIGENT UNIVERSITY EXAM HALL SEATING ALLOCATOR

Project Report

Submitted by:

Ahsan Raza, SP25-BDS-003

Muzzamil Ghaffar, SP25-BDS-034

Muhammad Irshad Khan, SP25-BDS-025

Sadaqat Hussain, SP25-BDS-040

Department of Computer Science, CUI, Islamabad.

Course: CSC262: Artificial Intelligence - I (Fall 2026)

Date: October 27, 2026

Project Category: Artificial Intelligence - Constraint Satisfaction Problem (CSP) / Search

Table of Contents

Abstract . . . 2

1. Introduction . . . 2

2. Proposed System . . . 2

3. Advantages/Benefits of Proposed System . . . 3

4. Scope . . . 3

Source Code and Output . . . 4

Source Code (Mid.py) . . . 4

Program Output . . . 6

5. Modules . . . 7

5.1 Module 1: Data Ingestion & Preprocessing Module . . . 7

5.2 Module 2: CSP Solver & Constraint Engine Module . . . 7

5.3 Module 3: Room & Infrastructure Manager Module . . . 7

5.4 Module 4: Visualization & Export Module . . . 7

6. System Limitations/Constraints . . . 7

7. Tools and Technologies . . . 8

8. Conclusion . . . 8

9. References . . . 8

Abstract

Allocating examination seating for universities handling hundreds or thousands of students is a complex combinatorial problem that is traditionally prone to human error, administrative bottlenecks, and academic integrity risks. This project presents an automated, AI-driven Intelligent University Exam Hall Seating Allocator modeled as a Constraint Satisfaction Problem (CSP) and solved via Backtracking Search. The system ingests student rosters, prioritizes hard accessibility constraints (ADA compliance), and enforces an anti-cheating isolation rule by preventing students of the same degree section or the same course paper from occupying orthogonally adjacent seats.

Designed and implemented in Python, the system processes and validates a multi-room seating allocation for 50 students across three examination halls, demonstrating correct, repeatable results and adherence to institutional academic standards. This report explains how each part of the proposal is realized in the submitted source file `Mid.py`, and includes the complete code and the output it produces.

1. Introduction

This report documents the implementation of the Intelligent University Exam Hall Seating Allocator. The problem is simple to state but hard to do by hand: seat 50 students in three halls so that students who share a degree section or a course paper never sit orthogonally next to each other, while students who need accessible seating are guaranteed an accessible seat. Doing this manually is slow and easy to get wrong.

The code solves it with a Constraint Satisfaction Problem (CSP) model. A single Python class, `ExamSeatingAllocator`, takes a Pandas DataFrame of students and a dictionary describing the halls, and returns a complete seating plan. Writing it exercised problem modeling, recursive state-space search, constraint checking, and tabular data handling with Pandas.

2. Proposed System

The CSP is formulated directly in the code as follows.

CSP element	In the proposal	In Mid.py
Variables	Students	The ordered list `all_ordered_students` (ADA students first, then the rest)
Domains	Physical seats	The list `available_seats`; each seat has an ID, room, row, column and ADA flag
Constraint 1	ADA accessibility	In `_is_safe`: an ADA student may only take a seat with `ada_compliant = True`
Constraint 2	Anti-cheating isolation	In `_is_safe`: no orthogonal neighbour (row + column distance of 1) in the same room with the same section or course
Search	Backtracking	Recursive `_backtrack`: assign, recurse, and on failure delete the assignment and try the next seat

The flow of `solve()` has three steps. Step 1 puts the ADA students at the front of the queue, because they have the most restricted domain (only row 1 of Hall_A), and assigning the most constrained

variables first avoids late dead ends. Step 2 builds the seat pool by looping over every room, row and column; a seat is marked ADA-compliant only if its hall is accessible and it is in row 1. Step 3 calls the backtracking engine.

Inside `_backtrack`, the base case is reached when every student has been placed (`index == len(students)`) and the function returns True. Otherwise it takes the current student, scans the seats in order, skips seats already in `self.allocation`, and for the first safe seat records the assignment and recurses on the next student. If the recursive call fails, the assignment is deleted (backtracking) and the next seat is tried. Constraints are checked at the moment of assignment; the code does not use forward checking or variable-ordering heuristics beyond the ADA-first ordering.

3. Advantages/Benefits of Proposed System

The following benefits are visible in the code and in its output.

1. Elimination of human error: the whole plan is generated by one call to `solve()`; all 50 students receive exactly one unique seat.

2. Academic integrity: `_is_safe` rejects any seat whose left, right, front or back neighbour shares the student's section or course. An independent check of the final plan found 0 such conflicts.

3. ADA accessibility: in the output all five ADA students (SP25-BDS-003, 015, 028, 039, 045) occupy Hall_A, row 1 (seats R1C1 to R1C5).

4. Speed: the run completes essentially instantly for 50 students, with no noticeable delay.

5. Multi-room scalability: halls are described by a dictionary (rows, columns, ADA flag); adding a hall is one extra entry, and the seat pool is built automatically.

6. Tabular reporting: the result is a Pandas table sorted by seat, ready to print for invigilators.

4. Scope

The implemented scope is a single Python file that (a) generates a synthetic roster of 50 students with IDs SP25-BDS-001 to SP25-BDS-050, names, sections (BDS-3A, 3B, 3C), courses (CSC262, MTH231, DSF201) and an ADA flag for students 3, 15, 28, 39 and 45; (b) defines three halls of 5 rows by 5 columns (75 seats in total), of which only Hall_A is ADA accessible; (c) runs the CSP solver; and (d) prints the allocation report.

The boundaries are the same as in the proposal: no OCR card scanning, no biometric attendance, and no invigilator duty rostering. Also outside the code are reading a real roster file (the data is generated in code) and graphical seating maps. The complete source code and the program output are given on the next pages.

Source Code and Output

Source Code (Mid.py)

```
import pandas as pd
import random

class ExamSeatingAllocator:
    def __init__(self, students_df, rooms_dict):
        self.students = students_df.to_dict(orient='records')
        self.rooms = rooms_dict
        self.allocation = {}

    def solve(self):
        # Step 1: Prioritize ADA students first
        ada_students = [s for s in self.students if s['ada_needed']]
        normal_students = [s for s in self.students if not s['ada_needed']]
        all_ordered_students = ada_students + normal_students

        # Step 2: Build available seat pool across rooms
        available_seats = []
        for room_name, r_info in self.rooms.items():
            for r in range(r_info['rows']):
                for c in range(r_info['cols']):
                    seat_id = f"{room_name}-R{r+1}C{c+1}"
                    available_seats.append({
                        'seat_id': seat_id,
                        'room': room_name,
                        'row': r+1,
                        'col': c+1,
                        # ADA students prefer ground floor / row 1 of accessible rooms
                        'ada_compliant': r_info['ada_accessible'] and (r == 0)
                    })

        # Step 3: Run Backtracking Engine
        return self._backtrack(all_ordered_students, 0, available_seats)

    def _is_safe(self, student, seat, assigned_seats):
        # Hard constraint: ADA check
        if student['ada_needed'] and not seat['ada_compliant']:
            return False

        # Soft anti-cheating constraint: Orthogonal adjacency check
        for s_id, assigned_student in assigned_seats.items():
            if assigned_student['room'] == seat['room']:
                d_row = abs(assigned_student['row'] - seat['row'])
                d_col = abs(assigned_student['col'] - seat['col'])

                if d_row + d_col == 1:
                    if (assigned_student['section'] == student['section'] or
                        assigned_student['course_code'] == student['course_code']):
                        return False
        return True

    def _backtrack(self, students, index, seats):
        if index == len(students):
            return True

        current_student = students[index]
        for seat in seats:
            if seat['seat_id'] not in self.allocation:
                if self._is_safe(current_student, seat, self.allocation):
                    seat_record = {**current_student, 'room': seat['room'], 'row': seat['row'], 'col': seat['col']}
                    self.allocation[seat['seat_id']] = seat_record

                    if self._backtrack(students, index + 1, seats):
                        return True

                    del self.allocation[seat['seat_id']]

        return False

if __name__ == "__main__":
    random.seed(42)
    first_names = ['Ali', 'Fatima', 'Bilal', 'Ayesha', 'Usman', 'Zainab', 'Hamza', 'Hira', 'Talha', 'Sana']
```

```
last_names = ['Khan', 'Noor', 'Ahmed', 'Malik', 'Tariq', 'Bibi', 'Ali', 'Amin', 'Zahid', 'Mir']
sections = ['BDS-3A', 'BDS-3B', 'BDS-3C']
courses = ['CSC262', 'MTH231', 'DSF201']

# Generate 50 realistic student records with 5 ADA students
students_list = []
for i in range(1, 51):
    students_list.append({
        'student_id': f"SP25-BDS-{i:03d}",
        'name': f"{random.choice(first_names)} {random.choice(last_names)}",
        'section': random.choice(sections),
        'course_code': random.choice(courses),
        'ada_needed': True if i in [3, 15, 28, 39, 45] else False
    })

df = pd.DataFrame(students_list)

# 3 Rooms with 25 seats each (75 total seats for 50 students to guarantee constraint satisfaction)
rooms = {
    'Hall_A': {'capacity': 25, 'rows': 5, 'cols': 5, 'ada_accessible': True},
    'Hall_B': {'capacity': 25, 'rows': 5, 'cols': 5, 'ada_accessible': False},
    'Hall_C': {'capacity': 25, 'rows': 5, 'cols': 5, 'ada_accessible': False}
}

print("Running CSP Exam Seating Allocator for 50 Students...")
allocator = ExamSeatingAllocator(df, rooms)

if allocator.solve():
    print("\nSUCCESS: Seating arrangement generated successfully!\n")
    report_data = []
    for seat, details in sorted(allocator.allocation.items()):
        report_data.append({
            'Assigned Seat': seat,
            'Student ID': details['student_id'],
            'Name': details['name'],
            'Section': details['section'],
            'Course': details['course_code'],
            'ADA Status': 'Yes (Reserved)' if details['ada_needed'] else 'Standard'
        })
    report_df = pd.DataFrame(report_data)

    # Display the entire DataFrame without truncation
    pd.set_option('display.max_rows', None)
    print(report_df.to_string(index=False))

    print(f"\nStatus: Successfully generated and verified seating charts for all {len(report_df)} students. Ready
else:
    print("FAILED: Could not satisfy constraints.")
```

Program Output

The output below was produced by running `Mid.py` with Python 3.11. Running it again gives the same table because the random seed is fixed to 42.

```
Running CSP Exam Seating Allocator for 50 Students...

SUCCESS: Seating arrangement generated successfully!

Assigned Seat   Student ID         Name Section Course     ADA Status
  Hall_A-R1C1 SP25-BDS-015    Sana Bibi  BDS-3C CSC262 Yes (Reserved)
  Hall_A-R1C2 SP25-BDS-039  Zainab Noor  BDS-3B MTH231 Yes (Reserved)
  Hall_A-R1C3 SP25-BDS-003 Fatima Zahid  BDS-3A DSF201 Yes (Reserved)
  Hall_A-R1C4 SP25-BDS-028     Hira Ali  BDS-3C MTH231 Yes (Reserved)
  Hall_A-R1C5 SP25-BDS-045    Sana Bibi  BDS-3B CSC262 Yes (Reserved)
  Hall_A-R2C1 SP25-BDS-002 Ayesha Malik  BDS-3A DSF201       Standard
  Hall_A-R2C3 SP25-BDS-001  Fatima Khan  BDS-3C MTH231       Standard
  Hall_A-R2C4 SP25-BDS-004   Hamza Khan  BDS-3A CSC262       Standard
  Hall_A-R2C5 SP25-BDS-005 Ayesha Malik  BDS-3C DSF201       Standard
  Hall_A-R3C1 SP25-BDS-011   Hamza Noor  BDS-3B MTH231       Standard
  Hall_A-R3C2 SP25-BDS-006    Ali Zahid  BDS-3A DSF201       Standard
  Hall_A-R3C3 SP25-BDS-009   Hamza Bibi  BDS-3B CSC262       Standard
  Hall_A-R3C4 SP25-BDS-020 Ayesha Tariq  BDS-3C DSF201       Standard
  Hall_A-R3C5 SP25-BDS-007    Talha Ali  BDS-3A MTH231       Standard
  Hall_A-R4C1 SP25-BDS-008   Sana Tariq  BDS-3A CSC262       Standard
  Hall_A-R4C2 SP25-BDS-019 Zainab Ahmed  BDS-3B MTH231       Standard
  Hall_A-R4C4 SP25-BDS-010  Ayesha Bibi  BDS-3A CSC262       Standard
  Hall_A-R4C5 SP25-BDS-014 Fatima Zahid  BDS-3B DSF201       Standard
  Hall_A-R5C1 SP25-BDS-024  Talha Malik  BDS-3C MTH231       Standard
  Hall_A-R5C2 SP25-BDS-012   Sana Tariq  BDS-3A DSF201       Standard
  Hall_A-R5C3 SP25-BDS-016  Fatima Khan  BDS-3C CSC262       Standard
  Hall_A-R5C4 SP25-BDS-018  Hamza Tariq  BDS-3B DSF201       Standard
  Hall_A-R5C5 SP25-BDS-013   Hira Zahid  BDS-3A MTH231       Standard
  Hall_B-R1C1 SP25-BDS-017   Usman Noor  BDS-3A CSC262       Standard
  Hall_B-R1C2 SP25-BDS-030  Talha Zahid  BDS-3B DSF201       Standard
  Hall_B-R1C3 SP25-BDS-021   Fatima Mir  BDS-3C CSC262       Standard
  Hall_B-R1C4 SP25-BDS-022  Talha Malik  BDS-3A MTH231       Standard
  Hall_B-R1C5 SP25-BDS-023  Hamza Tariq  BDS-3C DSF201       Standard
  Hall_B-R2C2 SP25-BDS-025    Ali Malik  BDS-3A MTH231       Standard
  Hall_B-R2C3 SP25-BDS-038 Fatima Zahid  BDS-3B DSF201       Standard
  Hall_B-R2C5 SP25-BDS-026  Hamza Tariq  BDS-3A CSC262       Standard
  Hall_B-R3C1 SP25-BDS-027    Sana Bibi  BDS-3A DSF201       Standard
  Hall_B-R3C3 SP25-BDS-029  Bilal Tariq  BDS-3A CSC262       Standard
  Hall_B-R3C4 SP25-BDS-031     Sana Ali  BDS-3C MTH231       Standard
  Hall_B-R3C5 SP25-BDS-050  Talha Ahmed  BDS-3B DSF201       Standard
  Hall_B-R4C1 SP25-BDS-034  Bilal Ahmed  BDS-3C MTH231       Standard
  Hall_B-R4C2 SP25-BDS-032 Zainab Malik  BDS-3A DSF201       Standard
  Hall_B-R4C3 SP25-BDS-035    Sana Noor  BDS-3B MTH231       Standard
  Hall_B-R4C4 SP25-BDS-033    Hira Noor  BDS-3A CSC262       Standard
  Hall_B-R4C5 SP25-BDS-036    Sana Amin  BDS-3C MTH231       Standard
  Hall_B-R5C1 SP25-BDS-040   Bilal Amin  BDS-3A DSF201       Standard
  Hall_B-R5C2 SP25-BDS-046  Fatima Bibi  BDS-3B CSC262       Standard
  Hall_B-R5C3 SP25-BDS-037   Talha Khan  BDS-3C DSF201       Standard
  Hall_B-R5C5 SP25-BDS-041  Usman Zahid  BDS-3A DSF201       Standard
  Hall_C-R1C1 SP25-BDS-042 Fatima Tariq  BDS-3C DSF201       Standard
  Hall_C-R1C2 SP25-BDS-043   Sana Malik  BDS-3A MTH231       Standard
  Hall_C-R1C3 SP25-BDS-044  Bilal Zahid  BDS-3C CSC262       Standard
  Hall_C-R1C4 SP25-BDS-048  Fatima Amin  BDS-3A DSF201       Standard
  Hall_C-R1C5 SP25-BDS-047    Ali Malik  BDS-3C CSC262       Standard
  Hall_C-R2C4 SP25-BDS-049  Bilal Ahmed  BDS-3C MTH231       Standard

Status: Successfully generated and verified seating charts for all 50 students. Ready for presentation.
```

Table 2: Verification of the output

Check on the generated allocation	Result
Students seated / total	50 / 50, each with a unique seat
Seats used per hall	Hall_A: 23, Hall_B: 21, Hall_C: 6 (25 seats each)
ADA students in accessible seats	5 of 5, all in Hall_A row 1
Orthogonal same-section / same-course neighbours	0 found

5. Modules

The four modules of the proposal map onto the code as follows.

5.1 Module 1: Data Ingestion & Preprocessing Module

Code: the data-generation loop in the `__main__` block, `__init__`, and Step 1 of `solve()`. The main block builds a list of 50 dictionaries (ID, name, section, course, ADA flag) using the `random` module with seed 42 and turns it into a Pandas DataFrame. The constructor converts the DataFrame to a list of records with `to_dict(orient='records')`. Step 1 of `solve()` uses two list comprehensions to split students into ADA and normal groups and joins them as `ada_students + normal_students`, so ADA students are always allocated first. Note that the code does not itself check for missing values or duplicate IDs; the generated IDs are unique by construction.

5.2 Module 2: CSP Solver & Constraint Engine Module

Code: the methods `_is_safe` and `_backtrack`. `_is_safe(student, seat, assigned_seats)` first applies the ADA rule: it returns False if the student needs an ADA seat and the seat is not compliant. It then loops over all assigned students in the same room, computes the row and column differences, and when their sum is 1 (directly adjacent) it returns False if the section or course code matches. `_backtrack` performs the depth-first search described in Section 2: choose the student at the current index, try each free safe seat, recurse, and undo with `del self.allocation[seat_id]` on failure. The dictionary `self.allocation` is the state of the search and the final answer.

5.3 Module 3: Room & Infrastructure Manager Module

Code: the `rooms` dictionary in the main block and Step 2 of `solve()`. Each hall stores its capacity, rows, columns and `ada_accessible` flag; Hall_A is accessible and Halls B and C are not. Step 2 uses three nested loops (room, row, column) to create a seat record with an ID such as `Hall_A-R1C1`, its room, 1-based row and column, and `ada_compliant`, which is True only for row 1 of an accessible hall. This gives a pool of 75 seats for 50 students, which leaves spare seats and makes the constraints easy to satisfy.

5.4 Module 4: Visualization & Export Module

Code: the reporting block after `solve()` returns True. It loops over `sorted(allocator.allocation.items())` so seats appear in order, builds a list of dictionaries with the columns Assigned Seat, Student ID, Name, Section, Course and ADA Status (shown as 'Yes (Reserved)' or 'Standard'), converts it to a DataFrame, and prints it with `to_string(index=False)` and `display.max_rows` set to None so that all 50 rows appear. A closing status line reports the number of students seated. If no solution exists, the program prints a failure message instead.

6. System Limitations/Constraints

- Each hall is a fixed rectangular grid, and all halls are hard-coded in the `rooms` dictionary.
- ADA seats are limited to row 1 of accessible halls. Hall_A row 1 has 5 seats, so this run can accommodate at most 5 ADA students, and it uses all 5.

- The search is plain backtracking in a fixed seat order with no forward checking or heuristics, so in dense or over-constrained cases (for example, a seat count close to the student count) the running time can grow exponentially. Having 75 seats for 50 students keeps this demonstration fast.
- The roster is synthetic and generated in code; the program does not yet read a real registration file or validate it for duplicates or missing values.
- The result is one valid arrangement, not an optimal or balanced one: the search fills earlier seats first, so Hall_A (23 students) and Hall_B (21) are nearly full while Hall_C has only 6.

7. Tools and Technologies

Table 1 lists what was used to build and run the program.

Table 1: Tools and Technologies for Project

Category	Tool / Technology	Version	Rationale
IDE	Visual Studio Code	Latest	Lightweight, robust code editing, running and debugging environment
Programming Language	Python	3.11	Core language for implementing the object-oriented CSP solver
Data Processing Library	Pandas	2.0+	Holds the student roster as a DataFrame and produces the tabular seating report
Randomization Utility	random (Python standard library)	Built-in	Generates a reproducible synthetic dataset of 50 students (seed 42)

8. Conclusion

The program models exam seating as a CSP and solves it with recursive backtracking search. In the run shown above, all 50 students were seated across three halls, all five ADA students received accessible seats in Hall_A row 1, and an independent check found no orthogonally adjacent students sharing a section or course. The design keeps the roster, room layout, constraint checking and reporting clearly separated, which makes it easy to extend. Future work could add forward checking, a real roster import, and a visual seating map.

9. References

[1] Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th Edition), Pearson, 2020.

[2] COMSATS University Islamabad, *CSC262 Artificial Intelligence Lab Manual and Guidelines*, Fall 2026.

[3] Python Software Foundation, *Pandas Documentation: Data Structures and Ingestion Utilities*, https://pandas.pydata.org/